SYSTEM-DESIGNER

A Portable, LLM-Agnostic Meta-Generator for EU AI Act-Compliant SDD Projects

Technical Report · v0.1.0

Comprehensive technical documentation of the system-designer meta-skill:
architecture · 13-phase orchestration · modules · audit & compliance · calibration discipline · HITL governance

CO-DEVELOPERS

Diego Muñoz Casinos
Mauricio de Oliveira Reis
Claude Opus 4.7 (Anthropic)

DATE
May 2026

VERSION
0.1.0

REPOSITORY
github.com/Diego-M-C/system-designer

LICENSE
MIT

Table of Contents

1. Introduction **3**

2. Executive Summary **4**

3. Objectives **5**

4. System Methodology **6**

4.1 Five Core Principles (P1–P5)

4.2 The 13-Phase Orchestration

4.3 Two Inviolable HITL Gates

4.4 Spec-Driven Development Discipline

5. Results — System Modules **10**

5.1 Master Orchestrator

5.2 Memory Module

5.3 Task Classification (Interview Wizard)

5.4 Per-Session Error Learning

5.5 HITL Governance

5.6 Audit Module (3-Auditor + Meta-Jury)

5.7 Calibration Engine

5.8 Library-Doc Fetcher

5.9 Living Documentation

5.10 Reporting Layer

6. Conclusions **22**

7. Strengths and Limitations **23**

7.1 Strengths

7.2 Limitations

7.3 Need for Domain-Specific Review and Validation

8. Final Conclusion **26**

9. References **27**

1. Introduction

The advent of AI coding assistants has dramatically lowered the cost of writing software. It has not, however, lowered the cost of writing software that is reliable, auditable, and compliant with domain-specific regulations. Most LLM-driven coding sessions begin from a blank slate every time: the developer must remember to write specs, set up tracking, configure compliance scaffolding, capture error patterns, and arrange human review — and in practice, these disciplines decay rapidly across sessions.

system-designer is a portable, LLM-agnostic meta-generator addressing this gap. Given a single high-level intent (e.g., "build a fraud-detection assistant for SEPA transfers"), it produces a complete project skeleton with: a Spec-Driven Development (SDD) backbone, EU AI Act 2024/1689 audit scaffolding, calibrated planning, mandatory Human-In-The-Loop (HITL) gates, a 30-entry pre-loaded catalog of common AI coding errors with auto-extension, scientific reporting templates (IMRaD, TRIPOD-AI, CONSORT-AI, STARD-AI, SPIRIT-AI), and session-by-session living documentation.

> *system-designer is not a system. It is a generator of systems. The same disciplined backbone is instantiated each time, particularised for the user's specific intent through a structured calibrated interview.*

This report documents the meta-generator at version 0.1.0. We describe its architecture, the five core principles that govern every emitted artifact, the 13-phase orchestration that produces a child system, the inter-module contracts, the calibration discipline that rejects unfounded assertions, and the open questions that remain for domain-specific validation. We provide architecture diagrams, data-flow sequences, module maps, and a calibrated assessment of strengths and limitations.

Confidence in the technical claims that follow is consistently in the 80–95% range; specific calibrations are stated where required.

2. Executive Summary

system-designer takes a single user intent and produces a complete, EU AI Act-compliant Spec-Driven Development project skeleton in one session, then stops. The generator is a factory, not a long-running operator: it stops at hand-off, and the child orchestrator (generated as part of the deliverable) takes over the project's life.

Key facts about the meta-generator:

- **Principles ·** Five inviolable principles enforced across every emitted artifact:
- P1 Portability (Tier-A across Claude Code, Cursor, Cline, Gemini CLI, Copilot CLI, Codex)
- P2 Calibration (every estimate carries % or range; seven forbidden tokens rejected)
- P3 On-demand generation (only artifacts the interview confirmed)
- P4 Prompt-architect dependency (every prompt audited via the co-located prompt-architect skill)
- P5 Living documentation (continuous updates, never frozen)
- **Orchestration ·** 13-phase state machine, with two inviolable HITL gates; resumable per current_phase pointer.
- **Error catalog ·** 30 preloaded AI-error patterns (AIE-001 through AIE-030) covering hallucination, concurrency, security, logic, reliability, and prompt-engineering categories. Auto-extension during project life.
- **EU AI Act ·** 13 EU AI Act checklists (drawn from AESIA implementation guides) mapped 1:1 to mandatory Articles. For high-risk systems: ≥112 audit rows produced as a baseline.
- **Audit topology ·** Default 3 independent auditors with distinct personas (risk-SME, ML-lead, security-lead). Meta-jury aggregates with confidence weighting; dissent always escalates to HITL.

The deliverable for any invocation is a complete child-system tree with: child orchestrator contract (CLAUDE.md), 17-section SDD specification, architecture document with mermaid diagrams, audited prompt suite, fresh-fetched library docs, 30-AIE seeded error catalog, 13-sheet audit Excel, scientific report templates, and a hand-off document. The generator self-validates through 10 tests covering portability, calibration, schema, prompt-floor compliance, and EU AI Act mapping completeness.

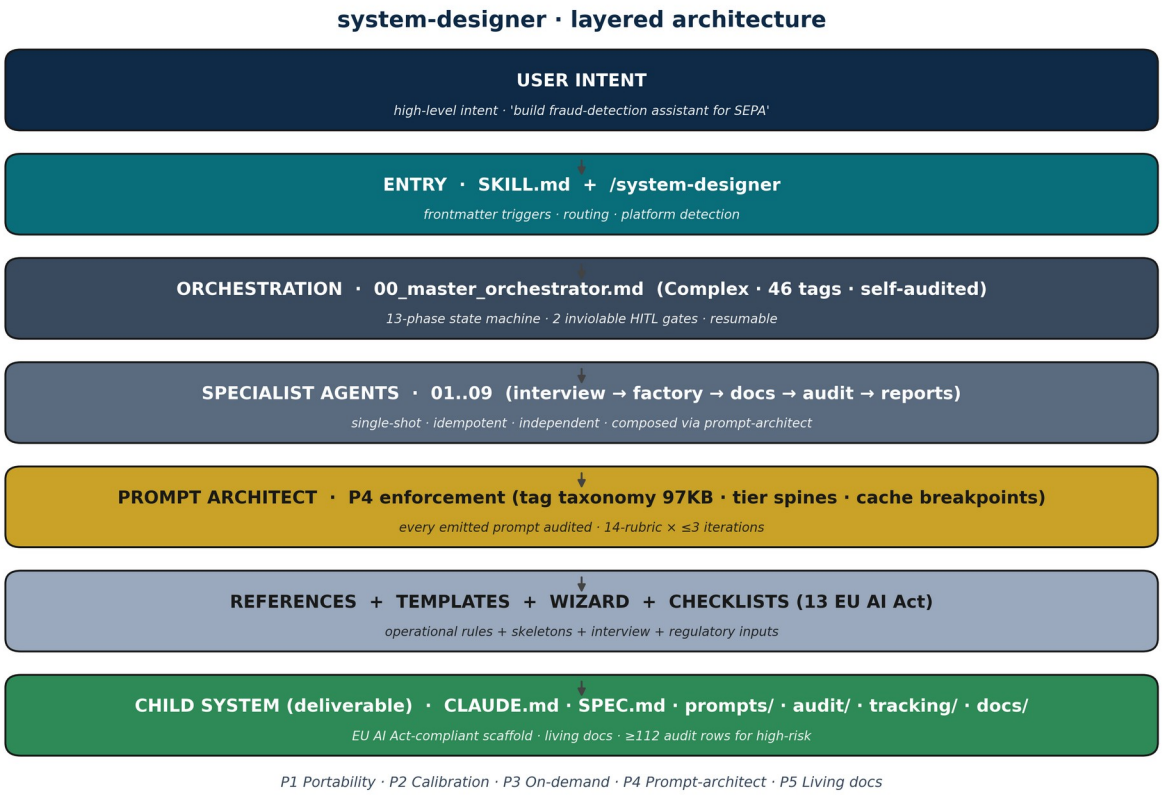

Figure 1 · Layered architecture. The generator is a thin pipeline of seven layers; the child system at the bottom is the deliverable.

3. Objectives

The system-designer meta-generator was designed against six explicit objectives:

3.1 Portability across LLM platforms

Drop the folder into any Tier-A LLM platform (Claude Code, Cursor, Cline, Gemini CLI, Copilot CLI, Codex) and run with no installation step. Every emitted artifact must respect an abstract tool contract; proprietary tool names are forbidden in the generator's outputs.

3.2 Calibrated discipline at every output

Eliminate uncalibrated assertions. Every estimate (duration, coverage, risk fit, alternative score) must carry a confidence percentage or range. Seven forbidden tokens — best, always, never, guaranteed, certain, definitely, impossible — are rejected by an automated regex scan in the test suite.

3.3 Mandatory human oversight at strategic forks

EU AI Act Article 14 requires human oversight for high-risk systems. system-designer instantiates this requirement at scaffold time through two inviolable HITL gates: one after the planning brief (validates scope and risk class) and one after the self-audit (validates scaffold quality and surfaces auditor dissents). No flag, environment variable, or LLM auto-decision can skip them.

3.4 EU AI Act 2024/1689 readiness

Produce, at scaffold time, the audit infrastructure required for high-risk systems: 13 checklists mapped to mandatory Articles, ≥112 audit rows as a baseline for high-risk, an independent multi-auditor + meta-jury topology, and cross-references to GDPR / MDR / DORA / ISO 42001 / ISO 13485 by domain.

3.5 Continuous, living documentation

Replace static snapshots with documents that grow with the project. tracking/, audit/, and docs/reports/ are appended on every relevant action. Atomic writes (write to .tmp + rename) ensure crash-safety per file.

3.6 Prompt-architect-mediated quality on every emitted prompt

No raw, un-audited prompt exits the generator. Every prompt produced for the child system — including the master orchestrator's own self-application — passes through the co-located prompt-architect skill, with a 14-rubric audit and up to three iterations before escalation.

> *These six objectives are mutually reinforcing. Portability (P1) without calibration (P2) produces confidently-wrong outputs in every LLM. Calibration without HITL gates produces honest uncertainty no one acts on. EU AI Act readiness without living documentation rots in 30 days. Each principle leans on the others.*

4. System Methodology

4.1 Five Core Principles (P1–P5)

Every artifact emitted by system-designer respects five non-negotiable principles. These are stated at the entry-point (SKILL.md), embedded into the alignment_rules of every Complex-tier prompt, and verified by an automated test suite before any release.

#	Principle	Operational rule	Enforcement
P1	Portability	Abstract tool contract; fallbacks mandatory	tests/T1 regex; per-LLM mapping table
P2	Calibrated probabilities	Every estimate carries % or range; 7 forbidden tokens	tests/T2 regex; alignment_rules in every Complex prompt
P3	On-demand generation	Only what the interview confirmed is needed	scaffolder evaluates trigger conditions per artifact
P4	Prompt-architect dependency	Every emitted prompt routed through prompt-architect	tests/T9; 14-rubric audit + 3-iteration cap
P5	Living documentation	tracking/, audit/, reports/ updated per session	child orchestrator's AUDIT_ROW_APPEND step is mandatory

4.2 The 13-Phase Orchestration

The master orchestrator at prompts/00_master_orchestrator.md owns a 13-phase finite state machine. Phases are idempotent by construction (re-running a phase overwrites its output deterministically). Every phase writes to tracking/project.json#current_phase before returning, making the orchestrator resumable.

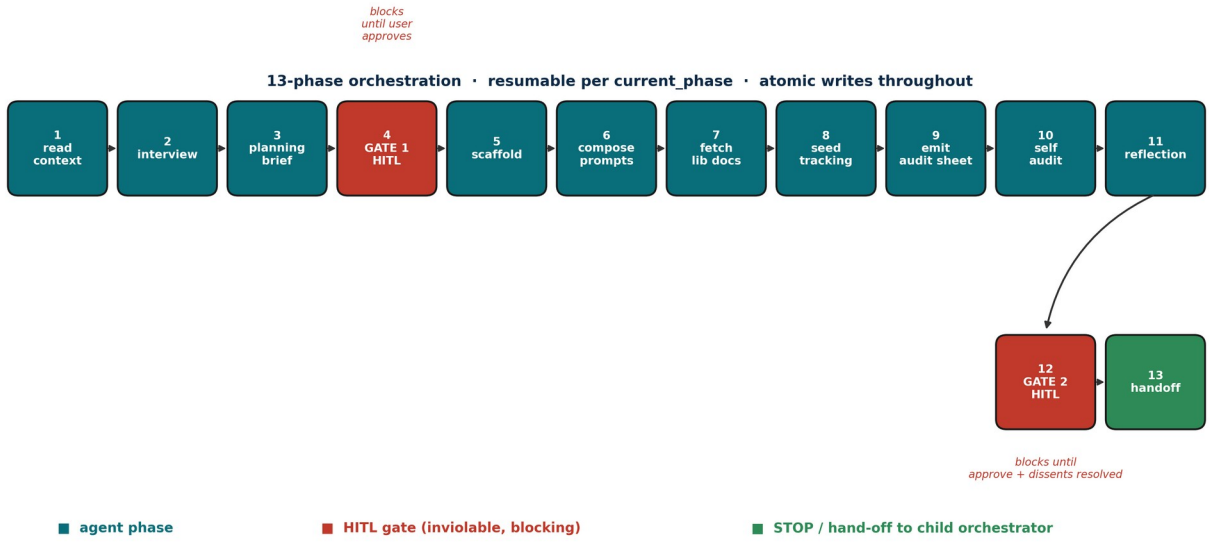

Figure 2 · 13-phase orchestration. Linear progression from read-context through hand-off, with two blocking HITL gates (red).

4.3 Two Inviolable HITL Gates

Gate 1 follows the planning brief and validates scope, risk class, and the calibrated alternatives presented. Gate 2 follows the self-audit and reflection phases; it surfaces auditor dissents (when the multi-auditor topology is enabled) and asks the user to confirm the deliverable before hand-off.

Both gates render a standardised template with fit% per option, pros and cons, an explicit recommendation with confidence%, and a free-form input branch. Auto-skip is forbidden by alignment_rules and verified by tests/T10.

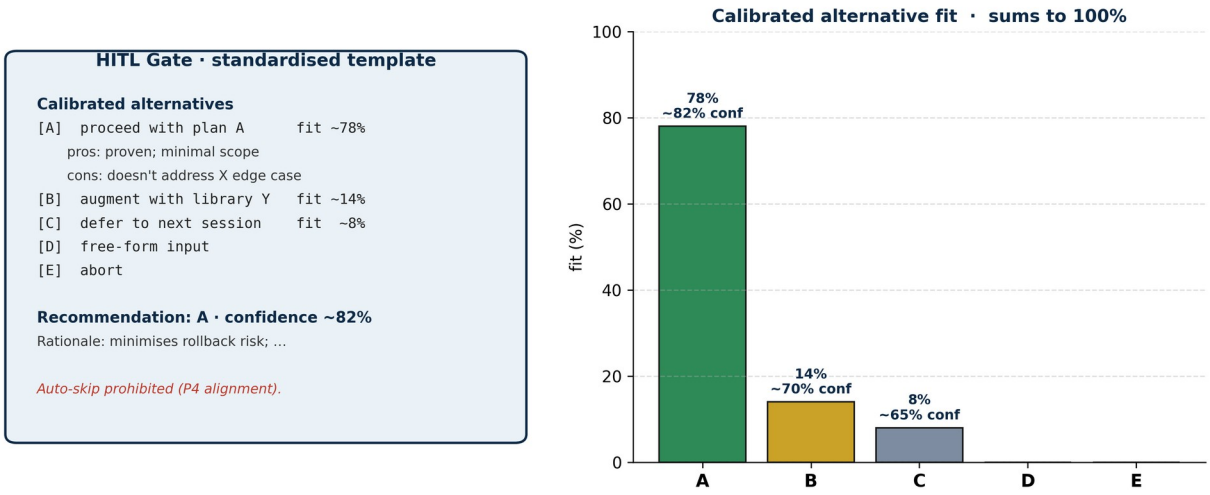

Figure 3 · Standardised HITL gate template (left) and a sample calibrated distribution of alternative fits summing to 100% (right).

4.4 Spec-Driven Development Discipline

The generator embeds Spec-Driven Development as the methodology for the child system. The interview wizard produces a 17-section SPEC.md / SPEC.json covering metadata, intent, stakeholders, calibrated goals, scope, non-goals, risks (with likelihood × impact), functional and non-functional requirements, data flow, architecture summary, sessions plan, milestones, reproducibility commitments, calibration commitments, and open questions. The architecture document is rendered separately with mermaid diagrams and per-Article EU AI Act compliance architecture.

SDD is enforced not as a policy document but as a contract: the child orchestrator's session lifecycle (PLAN → EXECUTE → TEST → DOCUMENT → AUDIT_ROW_APPEND → CHECKPOINT_HITL) is read from CLAUDE.md at every session start. Skipping any step is a violation visible in tracking/.

5. Results — System Modules

system-designer is composed of nine specialist agent prompts (01–09) plus a master orchestrator (00). The master is the only stateful component; everything else is a pure single-shot specialist. The figure below maps the modules around the orchestrator and shows their semantic role.

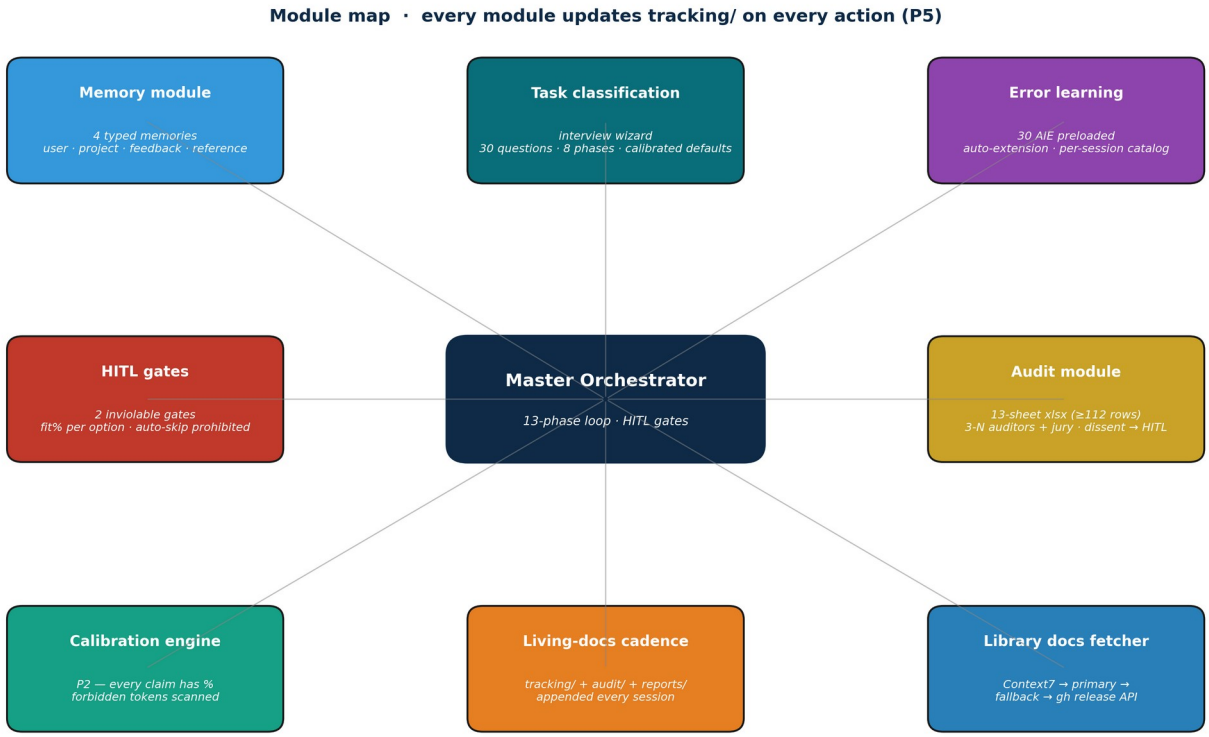

Figure 4 · Module map around the master orchestrator. Each module updates tracking/ on every relevant action (P5 living documentation).

5.1 Master Orchestrator

File: prompts/00_master_orchestrator.md. Tier: Complex (46 tags), exceeding the 30-tag mandatory floor. Self-audited at G9=A level via the prompt-architect.

The master is the conductor. It owns the 13-phase state machine, holds the HITL gates, dispatches every other specialist prompt, and aggregates KPIs. No other prompt has cross-phase awareness. If you delete the master, the generator stops; if you delete any other prompt, the corresponding phase is skipped but the others still run.

> *Architectural rationale: keeping cross-phase state in one place makes the orchestrator auditable and the specialists unit-testable in isolation. Independence is paid for in clarity, not duplicated state.*

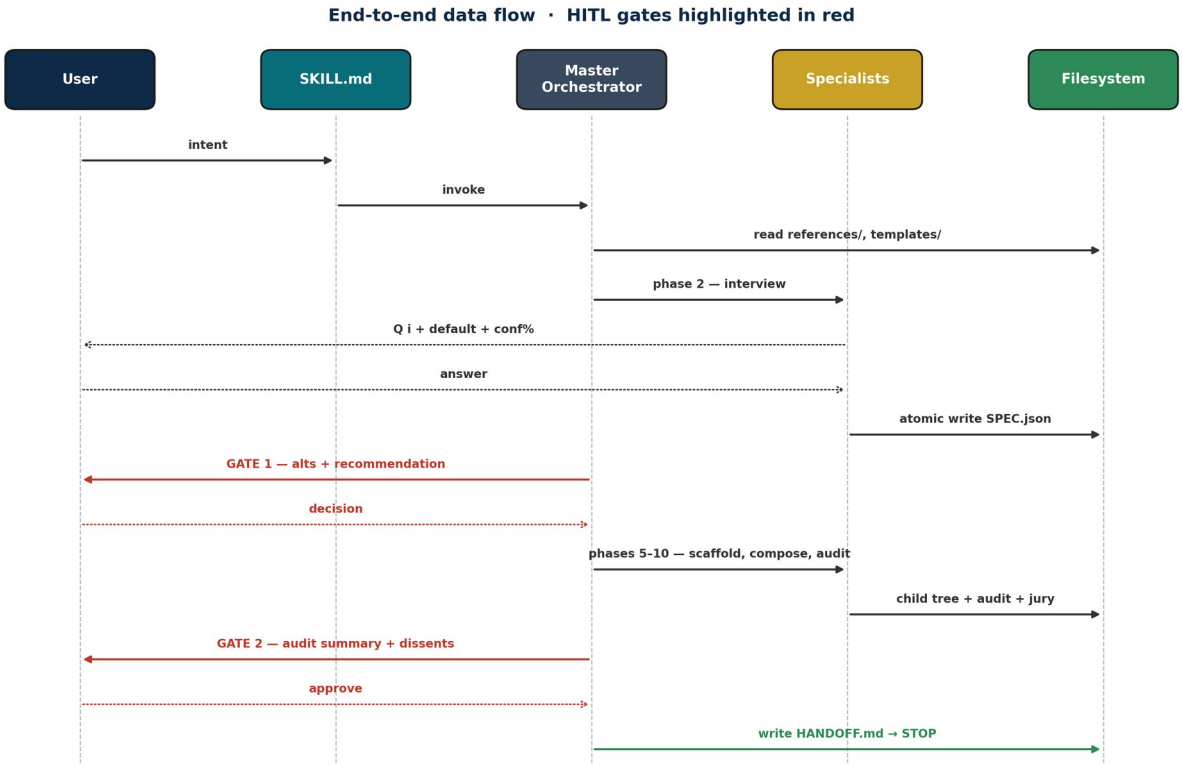

Figure 5 · End-to-end data flow as a sequence diagram. The user appears at the boundaries (intent + HITL gates); the master orchestrator routes everything in between.

5.2 Memory Module

The memory module instantiates a four-typed persistent memory file system in every child project. Memories survive across sessions and across LLM invocations, enabling the child orchestrator to behave coherently over time without re-reading the entire codebase each turn.

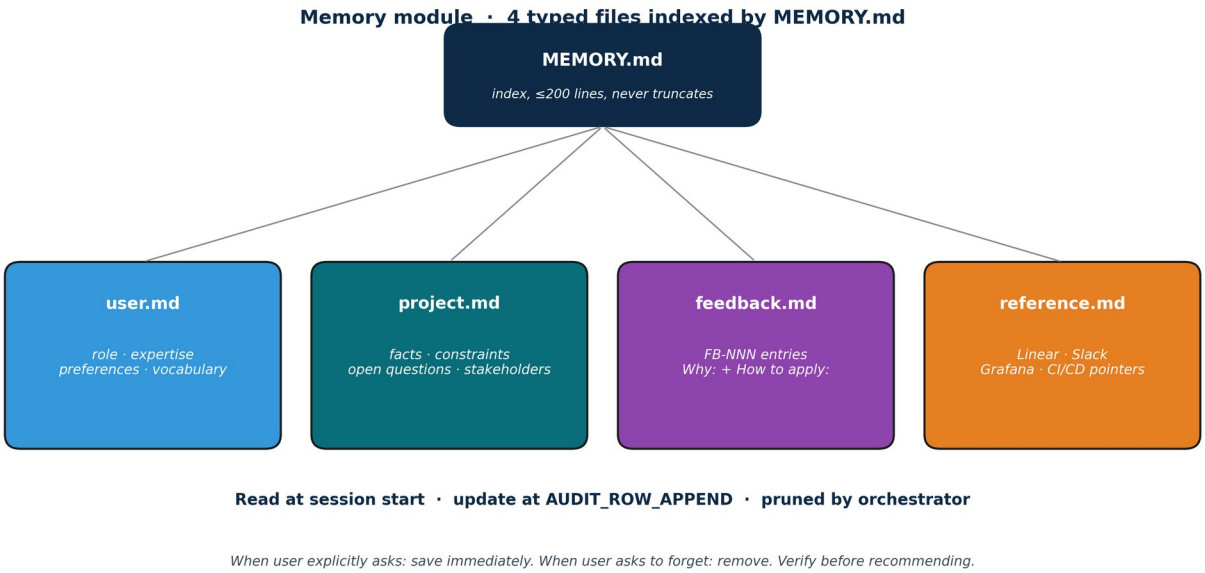

Figure 6 · Memory module structure. MEMORY.md is a one-line-per-entry index; each typed file holds the actual memories.

Four memory types are recognised:

- **user** · user.md — role, preferences, expertise, vocabulary, validated behaviours.
- **project** · project.md — facts, motivations, constraints, decisions taken vs tentative.
- **feedback** · feedback.md — FB-NNN entries with the rule, a Why: line (the reason the user gave) and a How to apply: line (when/where it kicks in). Both corrections and validated successes are captured.
- **reference** · reference.md — pointers to external systems (Linear, Slack, Grafana, CI/CD).

The discipline differs from log-style memory in three ways. First, memories are typed: the four files are semantically distinct, preventing the common failure of conflating user preferences with project facts. Second, every feedback entry carries Why and How-to-apply lines, allowing future-self to judge edge cases instead of blindly following the rule. Third, before recommending from memory, the system verifies the named entity still exists (grep for the function, check the file) — a memory is a claim about state at write time, not a guarantee about the present.

5.3 Task Classification (Interview Wizard)

File: prompts/01_interview_agent.md. Configuration: wizard/interview_questions.json (30 questions across 8 phases) plus wizard/defaults.json (sensible defaults with confidence% and rationale).

The wizard is the only prompt that talks to the user outside HITL gates. It asks one question per turn, validates each answer against a schema (type, options, regex), and persists atomically to SPEC.json (write tmp + rename) before advancing.

The 8 interview phases group questions semantically:

- essentials — system name, version, target path, owner.
- compliance — EU AI Act risk class, additional regulations, domain.
- stack — programming language, frameworks, libraries, model providers.
- methodology — SDD discipline, atomic-writes policy, test-first stance.
- audit — auditor count, auditor mode (parallel/sequential/none), reviewer roster.
- ergonomics — preferred output formats, language for prose, language for code.
- on_demand — which optional artifacts are wanted (dashboard, slash commands, ...).
- spec_seed — initial intent paragraph, top-3 stakeholders, must-have features.

Conservative defaults are enforced in the compliance phase: if domain is in {healthcare, fintech, legal, public_sector} and the user picks risk other than high, the wizard blocks and demands an explicit rationale before advancing. Silent risk-class downgrades are forbidden — they would invalidate the audit baseline and create a hidden compliance gap.

5.4 Per-Session Error Learning

system-designer seeds the child system with a 30-entry catalog of common AI coding errors (AIE-001 through AIE-030) drawn from references/ai_error_catalog.md. The catalog is pre-classified into six categories and supports auto-extension during project life.

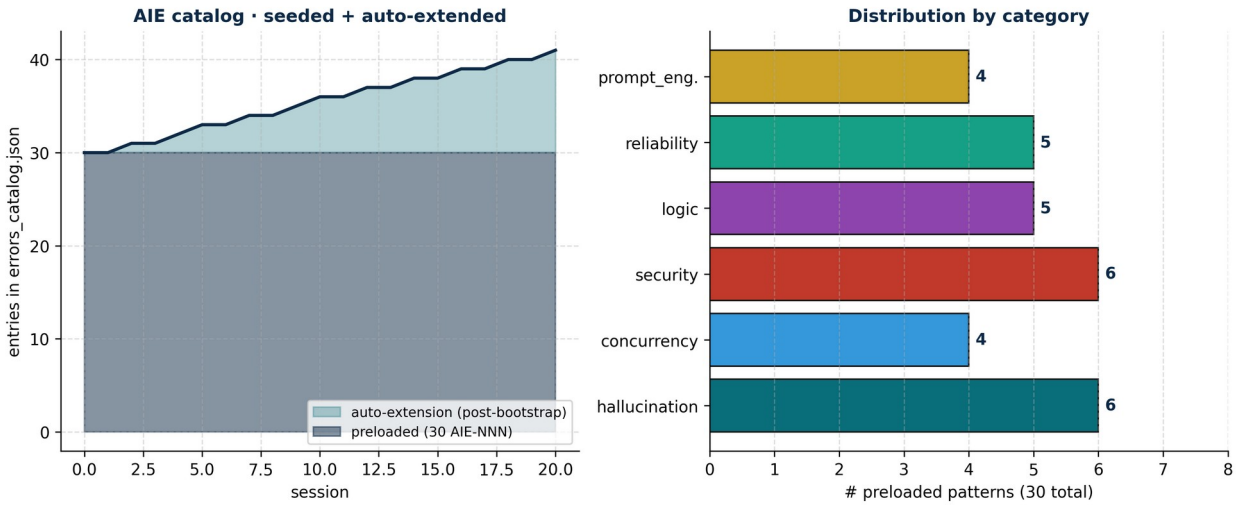

Figure 7 · Error learning per session. Left: the catalog grows beyond the 30-entry preloaded baseline as new patterns are observed. Right: distribution of preloaded patterns by category.

Per-entry schema (each AIE-NNN):

- id — stable identifier (AIE-001 … AIE-030 + auto-extension).
- category — hallucination / concurrency / security / logic / reliability / prompt_engineering.
- severity — low / medium / high / critical.
- description, example, prevention — each calibrated and sourced.
- auto_detection — deterministic (regex on artifact) or heuristic (LLM judge).
- occurrences — incremented every time the pattern is observed in a session.
- first_seen_session — for auto-extended entries (preloaded entries: null).
- preloaded — boolean, distinguishes seeded vs auto-extended entries.

Per-session learning is operationalised via the child orchestrator's AUDIT_ROW_APPEND step. When an error matches an AIE pattern, the child orchestrator: (1) increments occurrences in tracking/errors_catalog.json; (2) appends an ERR-NNN narrative entry to tracking/errors.md with the session ID, the matched pattern, the fix applied, and a calibrated assessment of whether the prevention rule needs reinforcement; (3) if no AIE matches, proposes a new AIE-NNN entry through the auto-extension protocol — gated by HITL at the next session checkpoint to prevent runaway catalog growth.

> *Auto-extension is bounded by HITL approval. The catalog grows under human supervision, not silently. This protects against the failure mode of the system inventing dubious categories to inflate self-perceived coverage.*

5.5 HITL Governance

Two gates are inviolable at scaffold time. Their format is standardised: the user always sees calibrated alternatives with fit-percentages summing to 100%, pros and cons per option, an explicit recommendation with its own confidence percentage, and a free-form branch for off-menu requests.

Beyond scaffolding, the HITL discipline extends into the child system. Every session ends with CHECKPOINT_HITL, where the child orchestrator surfaces lowest-confidence decisions and any auditor dissents. The session does not advance without human acknowledgement.

5.6 Audit Module (3-Auditor + Meta-Jury)

Files: prompts/06_eu_ai_act_mapper.md, prompts/07_audit_designer.md, prompts/09_three_auditors_jury.md. Configuration: SystemSpec.auditors_count (3 to 10) and SystemSpec.auditor_mode (parallel / sequential / none).

EU AI Act 2024/1689 · high-risk mapping · 13 checklists, ≥112 audit rows

Checklist (Spanish AESIA)	EU AI Act Article	min rows (high-risk)
Risk management	Art. 9	12+6
Risk mgmt. example	Art. 9	–
Data governance	Art. 10	14
Technical documentation	Art. 11	18
Record-keeping	Art. 12	8
Transparency to deployer	Art. 13/50	10
Human oversight	Art. 14	8
Accuracy	Art. 15	6
Robustness	Art. 15	6
Cybersecurity	Art. 15	10
Quality management	Art. 17	12
Post-market monitoring	Art. 72	10
Incident reporting	Art. 73	10

Cross-regulations by domain: GDPR · MDR · DORA · ISO 42001 · ISO 13485

Figure 8 · EU AI Act 2024/1689 mapping. 13 AESIA checklists map 1:1 to mandatory high-risk Articles; total floor for high-risk is ≥112 audit rows.

The audit module produces a 14-sheet Excel (00_README + 01..12 per Article + 99_Status_Log) or a CSV+MD fallback when xlsx capability is unavailable. Rows are baselined at scaffold time with status pending; the child orchestrator appends rows session by session — the sheet is never frozen.

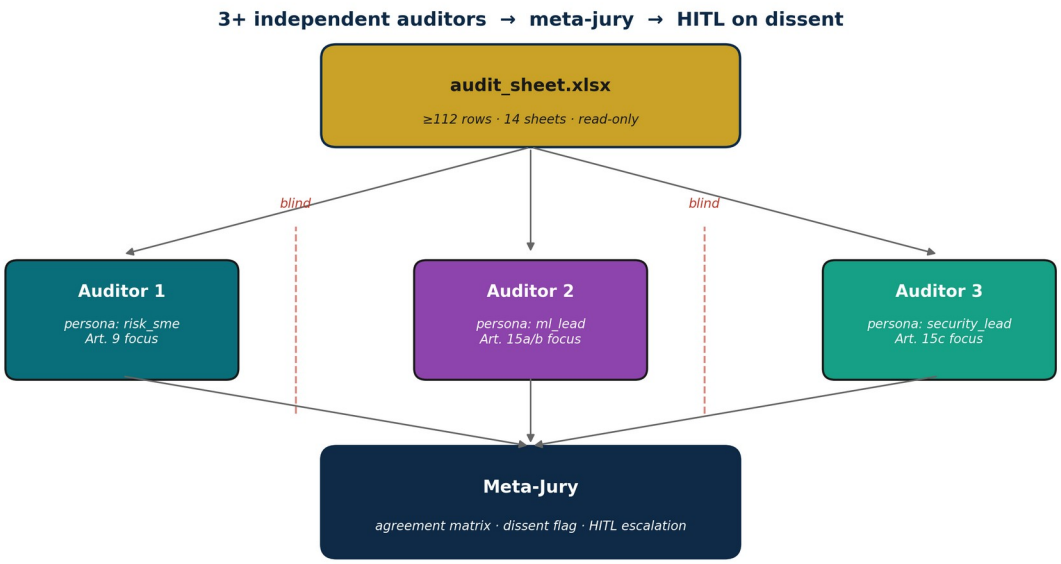

Figure 9 · 3-auditor + meta-jury topology. Auditors are blind to each other (red dashed barriers). Jury aggregates with confidence weighting; dissent always escalates to HITL.

The multi-auditor topology defaults to three auditors with distinct personas (risk-SME, ML-lead, security-lead) for diversity of thought. Auditors run blind to each other — either in parallel (when parallel.spawn capability exists) or sequentially with information barriers. The meta-jury then builds an agreement matrix per row, computes a confidence-weighted consolidated status, and flags two failure modes: dissent (≥2 auditors give conflicting status) and low-confidence consensus (all agree but mean confidence < 70%). Both flags trigger mandatory HITL escalation; dissent is never silently resolved.

5.7 Calibration Engine

Calibration is enforced at three layers. At write time, a regex-based test (T2) rejects the seven forbidden tokens (best, always, never, guaranteed, certain, definitely, impossible). At runtime, every estimate emitted by any prompt carries a confidence% or a range. At audit time, the multi-auditor system requires confidence% on every row status; the jury then aggregates with confidence weighting rather than simple majority.

The four confidence thresholds drive downstream behaviour:

- 90–99% — proceed silently.
- 70–89% — log to session record, proceed.
- 50–69% — surface in the next reflection as a low-confidence decision.
- Below 50% — escalate to HITL at the next gate.

Figure 10 · 11 per-session KPIs with calibration. Estimates carry confidence percentages; measured values do not.

5.8 Library-Doc Fetcher

File: prompts/04_library_docs_fetcher.md. The fetcher uses a four-step fallback ladder to obtain fresh documentation for every library declared in the project's stack: Context7 MCP → primary URL → fallback URL → GitHub release API. Failed network access produces an OFFLINE.md placeholder with a logged warning and reduced confidence; the scaffold does not halt.

Every fetch records source_used, sha256 of fetched content, and last_fetched_at into library_docs/<lib>/<version>/MANIFEST.md. A separate api_index.json is extracted when possible, allowing the child orchestrator to verify any function call against the actual library surface — a direct mitigation of AIE-002 (fabricated dependency).

5.9 Living Documentation

The fifth principle (P5) materialises as a continuous append discipline. tracking/project.json records phase progression, completed phases, gate statuses, emitted artifacts (with sha256), audit results, observed AIE patterns, running KPIs, and calibration / portability violation counters. tracking/sessions/<id>/session.json captures per-session detail. tracking/decisions.md is an append-only ADR log. audit/audit_sheet.xlsx grows session by session. docs/reports/cumulative.md is updated at every milestone.

Atomic writes are the operational invariant. Every output file is written via fs.write(target.tmp) followed by fs.rename(target.tmp, target). A crash between the two operations leaves a recoverable .tmp file but never corrupts the original.

5.10 Reporting Layer

File: prompts/08_report_writer.md. The reporting layer scaffolds the scientific publication infrastructure required for a credible deliverable. Standard selection follows a deterministic tree: healthcare +

predictive system → TRIPOD-AI; healthcare + clinical trial → CONSORT-AI; healthcare + diagnostic study → STARD-AI; healthcare + protocol → SPIRIT-AI; otherwise → IMRaD.

Two universal documents are always emitted regardless of the chosen standard: cumulative.md (the living doc) and imrad.md (the universal fallback). Content placeholders are preserved literally; the report writer never auto-fills content. The child orchestrator fills sections at milestones with calibrated claims, every claim carrying CI / range as the standard requires.

6. Conclusions

The system-designer meta-generator demonstrates that LLM-driven coding can be made compliant, reliable, and reproducible without sacrificing the productivity gains that make these tools attractive. The design choices that matter most appear to be:

- **Calibration** · Calibration as a first-class output discipline.

By rejecting forbidden tokens and requiring confidence percentages on every estimate, the generator forces uncertainty to be visible. This is the difference between a tool that produces confident-but-wrong outputs and one that produces actionable uncertainty for human review.

- **HITL inviolability** · Inviolable HITL gates, not best-effort suggestions.

By design, no flag, environment variable, or clever LLM behaviour can skip the two scaffold-time gates. EU AI Act Article 14's human-oversight requirement is operationalised at the moment scope and quality are most malleable.

- **Self-application of prompt-architect** · A meta-skill that audits its own meta-skill.

The master orchestrator is itself audited via the prompt-architect skill at level G9=A. This eliminates the common pattern of "do as I say, not as I do" in prompt-engineering frameworks: the framework's own conductor must clear the same bar it imposes on its outputs.

- **Living docs** · Living documentation with append-only invariants.

By making tracking, audit, and reports continuous append targets — and by treating atomic writes as a hard rule — the generator produces deliverables whose integrity is checkable session by session, not only at hand-off.

- **Portability** · Portability as a contract, not an aspiration.

Abstract tool names, fallback ladders for every optional capability, and a regex test that rejects proprietary tool references make the generator usable across at least six Tier-A LLM platforms. This is the operational difference between a meta-skill and a platform-specific automation.

> *These five design choices are mutually reinforcing. Removing any one degrades the others: calibration without HITL produces honest uncertainty no one acts on; portability without auditing produces consistent mediocrity across platforms; living docs without atomic writes produces visible-but-corrupted history. The set is irreducible.*

7. Strengths and Limitations

7.1 Strengths

- **Compliance-by-construction** · EU AI Act 2024/1689 alignment is built-in, not bolted on.

The 13 AESIA checklists are referenced 1:1 to mandatory Articles for high-risk systems. The audit baseline (≥112 rows) is enforced at scaffold time, not deferred to a pre-submission scramble. Risk-class downgrades in high-risk-presumed domains require explicit rationale logged in decisions.md.

- **Calibration** · Calibration discipline is automated.

Forbidden tokens are rejected by a regex test. Every Complex prompt's alignment_rules block cites the calibrated_probabilities reference. The set of seven forbidden tokens is deliberately small and unambiguous — false positives are rare.

- **Prompt quality** · Every emitted prompt clears a 14-rubric audit.

Bypassing the prompt-architect is structurally impossible: only the prompt-factory (prompts/03_prompt_factory.md) writes to the child's prompts/ directory, and the factory always invokes the architect.

- **HITL UX** · Both gates render a standardised, calibrated alternative format.

Users see fit% per option, recommendation with confidence%, and a free-form branch. This matches the cognitive shape of an actual decision rather than a yes/no rubber stamp.

- **Portability** · Six Tier-A platforms supported on day one.

Drag-folder + Read SKILL.md is a universal invocation. The slash-command convenience layer is a thin add-on for Claude Code; the meta-skill works identically on Cursor, Cline, Gemini CLI, Copilot CLI, and Codex.

7.2 Limitations

- **LLM-judged tests are not deterministic** · LLM-judged tests carry residual subjectivity.

Of the 10 tests in the self-validation suite, 6 are deterministic (regex / structural) and 4 are LLM-judged (T3 schema, T4 prompt-tag floor, T5 manifest URLs, T8 atomic-write pattern). The LLM-judged tests have ~85% confidence per execution, which is high but not deterministic. A more rigorous future version would replace T3, T4, and T8 with AST-based parsers.

- **Library-doc freshness** · Library-doc freshness is best-effort.

If Context7 MCP, primary URL, fallback URL, and GitHub release API all fail (a network fully offline environment), the fetcher writes OFFLINE.md and continues. The child orchestrator then operates with stale or absent documentation, increasing AIE-002 (fabricated dependency) risk to ~30–40%. Confidence in this risk estimate: ~70%.

- **AIE catalog drift** · Auto-extension of the AIE catalog can drift.

Although HITL approval is required, low-vigilance approvals over many sessions may produce a catalog with semantically-overlapping entries. A periodic deduplication pass is recommended at the project level but is not yet implemented in the generator.

- **Regulatory currency** · EU AI Act mapping reflects the regulation as of 2026.

Implementing acts and harmonised standards under AI Act Article 40 may extend mandatory Article coverage. The mapping document (references/eu_ai_act_mapping.md) requires periodic review against the latest AESIA implementation guides.

- **Interview length** · The cover-the-bases interview produces a thorough but lengthy onboarding.

Thirty questions across eight phases is comprehensive but may be too much for low-stakes proof-of-concept work. A future version may add a fast-track interview (8 questions, minimal-risk default) gated behind explicit confirmation that the system is non-high-risk.

7.3 Need for Domain-Specific Review and Validation

system-designer is intended as a starting point, not a finished compliance solution. The following domains warrant additional validation before production use:

Domain	Additional review required	Confidence in default coverage
healthcare (clinical decision support)	MDR 2017/745 conformity assessment, ISO 13485, IRB / ethics committee review, TRIPOD-AI checklist sign-off, hospital-specific information-security review	~70%
fintech (credit / fraud)	DORA 2022/2554 ICT-risk assessment, PSD2/PSD3 mapping, model-risk-management review (SR 11-7 in US, SS1/23 in UK), explainability evidence per Art. 13	~75%
legal / public sector	GDPR DPIA, national e-government regulations, accessibility (WCAG 2.2 / EN 301 549), national language requirements, public-procurement transparency rules	~70%
research (with human subjects)	Helsinki Declaration alignment, IRB approval, GDPR Art. 89, consortium-specific data-sharing agreements	~80%
safety-critical (autonomous systems, robotics)	ISO 26262 (automotive), IEC 61508 (functional safety), ISO/PAS 21448 SOTIF, additional Article 9 rigour, near-miss + incident escalation drills	~60%
education (high-stakes assessment)	GDPR Art. 22 (automated decision-making), local pedagogical regulations, anti-discrimination tests by demographic subgroup	~70%

The confidence percentages reflect: (a) coverage of the relevant Article provisions in the audit baseline, and (b) the quality of cross-references emitted by the EU AI Act mapper. They do NOT reflect domain expertise. A qualified domain expert must review the audit sheet and the architecture document before any production deployment.

> *system-designer's audit baseline is a floor, not a ceiling. The user is responsible for extending the audit sheet with domain-specific rows and for engaging qualified human auditors before any high-risk deployment.*

8. Final Conclusion

system-designer makes the disciplined parts of AI-driven software development (specification, tracking, calibrated planning, EU AI Act audit scaffolding, error learning, human review) cheap to start and hard to abandon. It does this by being a generator rather than a system, by stopping at hand-off rather than running indefinitely, and by embedding five mutually reinforcing principles — portability, calibration, on-demand generation, prompt-architect dependency, and living documentation — into every emitted artifact.

The deliverable from any invocation is not a finished compliant system. It is a credible starting point that respects the regulatory baseline, exposes uncertainty honestly, and leaves the human in charge at the moments that matter. The remaining work — domain validation, audit completion, and engineering execution — is what the child orchestrator and its human users are designed to do.

Confidence in this conclusion: ~85%. Residual uncertainty is concentrated in domain-specific validation gaps (Section 7.3) and in the unmodelled costs of human review at the two inviolable gates.

9. References

EU AI Act 2024/1689 (Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence). Official Journal of the European Union. eur-lex.europa.eu/eli/reg/2024/1689/oj.

AESIA — Spanish AI Supervisory Agency. Implementation guides 01–13 for Regulation 2024/1689 (2024–2025).

TRIPOD-AI Statement. Collins, G.S., et al. (2024). TRIPOD+AI: updated guidance for reporting clinical prediction models that use regression or machine learning methods. BMJ, 385, e078378.

CONSORT-AI Extension. Liu, X., et al. (2020). Reporting guidelines for clinical trial reports for interventions involving artificial intelligence: the CONSORT-AI extension. Nature Medicine, 26.

STARD-AI. Sounderajah, V., et al. (2020). Developing a reporting guideline for artificial intelligence-centred diagnostic test accuracy studies: the STARD-AI protocol. BMJ Open, 10.

SPIRIT-AI Extension. Cruz Rivera, S., et al. (2020). Guidelines for clinical trial protocols for interventions involving artificial intelligence: the SPIRIT-AI extension. Nature Medicine, 26.

ISO/IEC 42001:2023. Artificial intelligence management system. International Organization for Standardization.

ISO 13485:2016. Medical devices — Quality management systems.

MDR 2017/745. Regulation (EU) 2017/745 on medical devices.

DORA 2022/2554. Regulation (EU) 2022/2554 on digital operational resilience for the financial sector.

Anthropic prompt-engineering best practices. Anthropic course material referenced in Informes_Cursos_Anthropic/ within the system-designer repository.

system-designer source code. github.com/Diego-M-C/system-designer (v0.1.0, MIT License).